# ics-attacklab-2026 实验报告

### 观察栈帧 `input`

1. 看看 trace 部分，input 函数是谁调用的?

   是 `main` 调用的.

2. 看看 code:x86:64 部分，程序停下来的位置... 似乎不是 input 函数的第一条指令。停下来位置之前的三条指令是在做什么? $rbp 此刻指向的内存地址当中的值来自哪里?

   这三条指令先保存了 `rbp` 的值 (压栈)，然后将 `rsp` 的值赋给 `rbp`，接着为局部变量分配空间。此刻 `$rbp` 指向的内存地址中的值就是上一个 `rbp`，也就是 caller 函数 (main) 的栈帧基址。

3. 看看调用栈上 $rbp + 8 处，那里存的值是什么?（提示：运行 disas main 来查看它的汇编代码）

   是返回地址，就是调用完了以后要回到的指令。

4. 刚刚我们在 input 函数中读到的汇编代码似乎不能解释 $rbp + 8 处值的存在，我们需要找到它的来源。请你在 input 的调用者中找到 input 被调用的位置，在这里之前下一个断点，重新运行程序，并使用 si 命令从那里逐指令执行到我们的 input 断点，观察信息，回答是哪一行汇编代码在 $rbp + 8 处存入了值。

   `0x401acc <main+0009>  call  0x401977 <input>` 是这个 call 指令保存的。

5. 用同样的办法逐指令从 input 的 leave 指令之前处执行到 input 的调用者当中并观察每一步栈帧的变化，结合问题 2、问题 3 和 问题 4 回答 call、prologue、leave 和 ret 的作用。

   call: 返回地址压栈，跳到 callee.

   prologue: 保存 caller 栈帧基址 (以及一些寄存器)，分配 callee 空间.

   ret: 返回地址弹出，跳回 caller.

### 用不同长度的字符串输入测试 dark-calc

1. 用 20 和 32 个字符的字符串输入测试 dark-calc，观察程序的行为。

可以看出 20 个输入被截断到 15，但是 32 个导致了 segmentation fault.

Enter an expression (length up to 15): 12345678901234567890 Executing: echo $((123456789012345)) 123456789012345

Enter an expression (length up to 15): 123456789012345678901234567890 12 [1] 77222 segmentation fault (core dumped) ./dark-calc

2. 使用 GDB 在 input 函数中调用的 gets 函数后下一个断点，再次尝试输入并观察栈帧，看看超量的数据被写到了哪里?

传入的 char* pointer 地址在 0x0000721000800fd0. 这个是 input 函数里面开的局部 buffer 但是只有 16 个 bytes，更高的位置分别是 main 函数的栈帧基址和返回地址 (main 里面调用 input 以后的返回地址)。超出的数据会被写到这两个位置: 21 个 bytes 覆盖了 main 函数的栈帧地址，main 函数的 fbp 被破坏，但是由于一直在循环 input 和执行，没有报错; 33 个 bytes 则覆盖了返回地址，ret 的时候进入一个非法地址，导致 segmentation fault.

### 利用缓冲区漏洞执行./malware

eval 函数里面 `401a6b:  mov  $0x6030d0,%edi` 这个指令是把 0x6030d0 这个全局 buffer 里的东西输入 system 函数，system 函数执行这个字符串。所以我们只要想办法覆盖这个地址变成 `./malware` 就行了。回溯一下发现这个地址就是 gets 的输入，也就是我们只需要把 payload 构建成 `./malware` + padding + main rbp + padding + return address 就可以了。注意这里要防止 rbp 被覆盖成一些不对的东西（用正确的 main 函数的 rbp 覆盖，即 `0x721000800ff0`），然后把 input 的 return 地址变成刚刚那个 system 函数的开头的传参指令就行。也就是我们想办法跳过了和 echo 拼接的逻辑。

最终 payload

```
payload = b'./malware\0\0\0\0\0\0\0' + b'\xf0\x0f\x80\x00\x10\x72\x00\x00' +
 ↪  b'\x6b\x1a\x40\x00\x00\x00\x00'
```

这里我们只有 31 个字节去掉了因为 gets 会在最后加 \0.

### 用 GDB 检查 main 函数的机器码

1. 前三行汇编对应的机器码分别是什么?

   0x0000000000401ac3 <+0>: 55 push rbp

   0x0000000000401ac4 <+1>: 48 89 e5 mov rbp,rsp

   0x0000000000401ac7 <+4>: b8 00 00 00 00 mov eax,0x0

2. 这些机器码在内存中排列的方式是什么样的? 请你用文字或画一个图来说明。需要指出这三行指令分别对应的内存区间是什么。你可以在第一行指令处下一个断点，然后逐条指令执行，同时观察 rip 寄存器的值。

   就是一个一个字节存放，同时是变长的，而且 opcode 不用反转，从小到大一个一个字节读取。比如前三条分别的起始地址是 0x401ac3、0x401ac4、0x401ac7，对应的机器码分别是 55、48 89 e5、b8 00 00 00 00。这些机器码的字节就是向高地址方向一个一个生长的，但是 0 这个操作数需要反转虽然这里看不出来。三个指令的内存区间分别是: 0x401ac3-0x401ac3、0x401ac4-0x401ac6、0x401ac7-0x401acb。(闭区间)

### 任务：栈空间执行

主要的思路就是在 payload 的 buffer 里面写一个能够传入 edi 并且调用 grade_eval 这个函数的机器码。但是这里有一个 trick 就是如果你使用 call 会把 call 的下一个指令地址当成返回地址，这样 grade_eval ret 的时候回到这个地址，程序不能正确执行完。我们的思路是在机器码里面先改 edi 传参数 (这里应该是 4)，再 push 正确的返回地址 (main 函数 call 完 input 的下一条指令)，然后再 push grade_eval 的指令地址，然后 ret。这样第一次 ret 调用 grade_eval, grade_eval 里面的 ret 就会回到我们 push 的返回地址，程序就可以继续执行了。汇编应该是

```
mov edi, 4
push 0x401ad1
push 0x401a87
ret
```

正好 16 个字节。

最后 payload 的组成是 机器码 + main rbp + 栈上机器码所在地址.

```
payload = b'\xBF\x04\x00\x00\x00\x68\xD1\x1A\x40\x00\x68\x87\x1A\x40\x00\xC3' \
+ b'\xf0\x0f\x80\x00\x10\x72\x00\x00' + b'\xd0\x0f\x80\x00\x10\x72\x00'
```

## ROP

首先要找到 gadget，这里我看机器码有点头疼，参考了网上的博客 https://ypl.coffee/hidden-gadget/. 可以看到一个 gadget 是 `pop rdi; ret` ，地址是 401a32 - 401a33‘。恰好我们需要用 edi 传参数 4. 所以我们的思路就是先 ret 到这个 gadget，然后栈高地址方向放 4，这样 edi 就是 4 了，然后栈的再高我们就存放 grade_eval 的地址，然后再高存放 main 里返回地址 (input 的下一个地址)。

这里我遇到几个问题

1. 我们是否要保护 rbp? 实际上是不需要的因为即使 main 里面 rbp 没有被恢复，由于程序的逻辑是在 input 里面退出，没有地方用到 main 的 rbp。

2. 最开始的 junk 不能填写 0，这样会直接退出。

3. 这个问题最棘手，虽然我已经打印出对应的 cookie 了，但是后续 eval 调用 system 函数的时候会报错，导致程序不能健康退出。这是对齐问题，正常函数调用 call 的时候压入 8 字节返回地址，而我们是通过 ret，所以这边栈有一个错位。然后 glibc 的 do_system 里面有相关检查，所以就报错了。修复方法是在我们用 ret 跳 grade_eval 之前先 ret 跳一个 ret 的 gadget，这样就对齐了，这个 ret 的 gadget 能够正确到 grade_eval (这个 bug 太难找了)

最终 payload

```
payload = b'\x11' + b'\x00' * 23 + b'\x33\x1a\x40\x00\x00\x00\x00\x00' +
↪  b'\x04\x00\x00\x00\x00\x00\x00\x00' + b'\x77\x19\x40\x00\x00\x00\x00\x00' +
↪  b'\x78\x19\x40\x00\x00\x00\x00\x00' + b'\xc2\x19\x40\x00\x00\x00\x00'
```

## Canary

我们先啃一下汇编，发现这个 input 函数开了 288 字节局部空间，然后 canary 在 rbp - 8, rbp - 288 用 gets 写入我们的 payload，rbp - 32 这个位置在 get 前被设置成 0，get 后又被用于计算 `rax = rbp - 288 + 128 + len` len 就是 rbp - 32 的值。后续 rbp - 288 开始的 128 个字节被复制到 rax。这样我们自然能够想到我们可以用 gets 改变 len 位置的值，从而 rbp - 288 开始的 128 个字节可以复制到 main 的 canary 以上，具体来说可以从 main 的返回地址 开始覆盖，思路类似上一题，把 main 的 ret 变成上一题的 gadget，后续都和上一题一样。这样直到 main ret 之前都不会出现任何问题，canary 照样校验成功，ret 的时候就和上一题是一样的逻辑了。

这样 payload 的布局是 gadget + 4 + grade_eval 地址，然后 232 个垃圾，然后 十进制 200 (len). 实测不需要像上一题一样管对齐，反正最后都是 main 的 ret 环节了，让他继续正常运行没有任何意义。

**本次实验我一直执着于让进程正常退出，实际上我到现在才发现好像测试只检查是不是打印出来了 grade_eval 的输出 lol**

```
payload = b'\xa3\x18\x40\x00\x00\x00\x00\x00' \
          + b'\x04\x00\x00\x00\x00\x00\x00\x00' \
          + b'\x77\x16\x40\x00\x00\x00\x00\x00' \
          + b'A' * 232 \
          + b'\xc8\x00\x00\x00' \
```

Q: 为了进一步阻止这种利用方式，现代编译器会对栈上局部变量存储的数据位置进行重新排布，我们在这里通过一个结构体固定顺序才创造出了利用空间。想一想，应当如何重新排列这些局部变量，才能使上述攻击失效？为什么？

A: 对于这个题来说，直接把原来 rbp - 32 这个位置的 len 放到最低就行了，这样就写不到他，从而两个写行为（一个是 payload 写入，一个是 payload copy）都不能不破坏而越过 canary。更加 general 的 idea 是任何可写 buffer 都要靠近 canary，然后长度这种控制数据都放在低地址防止被利用。